

Análise da Pilha de Protocolos HTTP e TCP: Um Estudo de Tráfego de Rede Aplicado ao Sistema Fornos

Thiago Henrique Markendorf, Rafael Gustavo Kammler, Henrique Vieira

Departamento de Engenharias e Ciência da Computação
Universidade Regional Integrada do Alto Uruguai e das Missões (URI)
Caixa Postal 743 – 99.709-910 – Erechim, RS – Brazil
{ 109072, 043372, 108500 }@aluno.uricer.edu.br

Resumo

Este artigo apresenta um estudo técnico sobre a comunicação de dados e o tráfego de rede aplicado ao projeto integrador Fornos, um sistema web de gerenciamento de pedidos para panificadoras. O objetivo principal do relatório é mapear o fluxo de informações através da pilha de protocolos TCP/IP, conectando conceitos teóricos da disciplina de redes de computadores à prática da engenharia de software. A análise abrange a atuação da Camada de Transporte, destacando o mecanismo de confiabilidade do protocolo TCP, e detalha a Camada de Aplicação, evidenciando o uso dos métodos HTTP e cabeçalhos para o tráfego de regras de negócio e controle de acesso (RBAC). Adicionalmente, o estudo explora o isolamento lógico provido por redes virtuais containerizadas (Docker) para a comunicação interna com o banco de dados PostgreSQL. Por fim, o trabalho propõe uma reflexão sobre eficiência de rede, comparando o alto custo de tráfego do atual modelo de *HTTP Polling* com uma futura migração para o protocolo WebSocket. Conclui-se demonstrando a dependência intrínseca das aplicações *full-stack* em relação aos protocolos de comunicação, evidenciando que a compreensão da infraestrutura de redes é fundamental para a viabilidade e evolução contínua do *software*.

1 INTRODUÇÃO

A evolução das aplicações web modernas consolidou o modelo cliente-servidor como o paradigma central para a engenharia de software distribuída. Nesse cenário, a comunicação de dados eficiente e segura é garantida pela atuação conjunta de protocolos padronizados, estruturados em camadas, que abstraem a complexidade do tráfego físico. A pilha de protocolos TCP/IP (*Transmission Control Protocol/Internet Protocol*), operando em conjunto com o protocolo HTTP (*Hypertext Transfer Protocol*) na camada de aplicação, atua como a espinha dorsal da internet, permitindo que sistemas interajam de forma confiável, íntegra e escalável.

Inserido nesse contexto tecnológico, o projeto integrador denominado “Fornos” foi idealizado e desenvolvido como uma plataforma de gerenciamento de pedidos, voltada para o setor de panificação. Arquitetado sob o modelo *full-stack* com a utilização do *framework* Next.js, o sistema opera sobre uma infraestrutura integralmente containerizada via Docker, garantindo o isolamento dos serviços. A aplicação lida com regras de negócio operacionais complexas, com

destaque para o controle de acesso baseado em perfis (RBAC), o gerenciamento autônomo de múltiplos estabelecimentos filiais e um sistema de atualizações síncronas projetado para o monitoramento contínuo do fluxo de pedidos.

Embora o desenvolvimento do Fornos concentre sua complexidade arquitetural nas camadas superiores da engenharia de software, a totalidade de sua operabilidade é estritamente dependente da infraestrutura de redes subjacente. Cada interação realizada na interface da plataforma — seja o carregamento inicial do catálogo de produtos por um cliente, a submissão de um formulário de configuração pelo gerente da loja, ou a requisição interna para persistência de dados no banco relacional PostgreSQL — traduz-se no encapsulamento de informações, no estabelecimento de conexões lógicas e na transmissão contínua de pacotes de dados.

Diante do exposto, o presente relatório técnico tem como objetivo realizar um estudo aplicado sobre a comunicação de dados no ecossistema da aplicação Fornos. O trabalho busca mapear e analisar o fluxo de informações que transita entre o navegador do usuário, o servidor da aplicação e a base de dados, evidenciando a materialização dos conceitos teóricos de redes e telecomunicações nas operações diárias de um software em ambiente de produção.

Para atingir tal propósito, este artigo está estruturado de modo a conectar a teoria à prática de engenharia de software. A Seção 2 apresenta a fundamentação teórica pertinente, focando na abstração das camadas do Modelo OSI e da pilha TCP/IP. A Seção 3 detalha a atuação primordial da Camada de Transporte, evidenciando o mecanismo de confiabilidade do protocolo TCP. A Seção 4 analisa o tráfego gerado pela Camada de Aplicação, dissecando o uso dos métodos HTTP e cabeçalhos de segurança na operabilidade da plataforma. A Seção 5 expõe a mecânica da rede interna de contêineres entre o servidor e o banco de dados. A Seção 6 apresenta uma análise de tráfego comparativa entre o atual modelo de *HTTP Polling* e a futura implementação do protocolo WebSocket. Por fim, a Seção 7 reúne as considerações finais acerca do estudo realizado.

2 FUNDAMENTAÇÃO TEÓRICA: A PILHA TCP/IP E O MODELO OSI

Para que a comunicação entre sistemas heterogêneos seja viável em escala global, a arquitetura de redes de computadores baseia-se em modelos de referência estratificados em camadas. Conforme elucidam [2], essa abordagem modular permite que problemas complexos de comunicação sejam divididos em partes menores e mais gerenciáveis, onde cada camada oferece serviços específicos para a camada imediatamente superior, ocultando os detalhes de implementação das camadas inferiores.

O modelo de referência OSI (*Open Systems Interconnection*), padronizado pela ISO (*International Organization for Standardization*), é o principal arcabouço conceitual utilizado para descrever a comunicação de rede. Ele divide o processo de comunicação em sete camadas: Física, Enlace, Rede, Transporte, Sessão, Apresentação e Aplicação. O tráfego de dados neste modelo segue o princípio do encapsulamento: quando um usuário interage com um software,

os dados são gerados na camada de Aplicação e descem pelas camadas sucessivas até a camada Física, recebendo cabeçalhos de controle (*headers*) em cada etapa antes de serem transmitidos pelo meio físico [1].

Embora o Modelo OSI seja a principal referência didática e teórica, a arquitetura que efetivamente governa o tráfego da Internet contemporânea, e sobre a qual o sistema Fornos opera, é a pilha de protocolos TCP/IP. Diferentemente das sete camadas estritas do OSI, o modelo TCP/IP condensa as funções em quatro (ou cinco, dependendo da abordagem bibliográfica) camadas principais: Aplicação, Transporte, Rede (ou Internet) e Acesso à Rede (que engloba Enlace e Física).

Neste contexto, o roteamento de pacotes e a resolução de endereços IP (*Internet Protocol*) ocorrem na camada de Rede, garantindo que os dados naveguem pela infraestrutura e alcancem o servidor correto. Contudo, do ponto de vista do desenvolvimento de *software* e da engenharia da plataforma Fornos, a interface direta do código com a rede ocorre majoritariamente nas duas camadas superiores da pilha TCP/IP.

A camada de Transporte é responsável por estabelecer a comunicação fim a fim entre o navegador do usuário e o servidor de hospedagem, sendo o TCP (*Transmission Control Protocol*) o protocolo primário utilizado para garantir que os dados não sejam perdidos ou corrompidos durante o trajeto. Imediatamente acima, a camada de Aplicação provê a semântica da comunicação, utilizando o protocolo HTTP para trafegar as regras de negócio, as chamadas de banco de dados e a interface gráfica do sistema. As seções subsequentes detalharão a atuação prática dessas duas camadas na arquitetura do projeto integrador.

3 A CAMADA DE TRANSPORTE: CONFIABILIDADE COM O TCP

A Camada de Transporte atua como o elo entre os processos de aplicação e a infraestrutura de roteamento de rede, provendo a comunicação lógica fim a fim. No ecossistema do sistema Fornos, esta responsabilidade é delegada ao protocolo TCP (*Transmission Control Protocol*), selecionado fundamentalmente por sua característica orientada à conexão e pela garantia estrita de confiabilidade na entrega dos dados. Diferentemente de protocolos não orientados à conexão (como o UDP), o TCP implementa mecanismos sofisticados de controle de erro, controle de fluxo e contenção de congestionamento, essenciais para transações comerciais.

O estabelecimento dessa comunicação confiável ocorre através de um processo preliminar de negociação denominado *Three-Way Handshake* (Aperto de mão em três vias). Conforme detalha [4], antes que qualquer dado da aplicação seja trafegado, o host cliente envia um segmento de sincronização (SYN) ao servidor. O servidor, ao aceitar a requisição, responde com um segmento de reconhecimento e sincronização (SYN-ACK). Por fim, o cliente confirma o recebimento da resposta (ACK), estabelecendo o canal lógico, o que consome exatamente 1,5 RTT (*Round-Trip Time*) de latência inicial.

Na prática operacional do projeto integrador, este *handshake* atua como um pré-requisito

absoluto e invisível ao usuário final. Quando um cliente acessa a interface da plataforma para visualizar o catálogo de uma padaria, ou quando o gerente autenticado submete uma atualização de estoque, o navegador e o servidor de hospedagem do Fornos (operando sob a arquitetura Next.js) já executaram essa negociação nos bastidores.

Essa escolha arquitetônica garante que a integridade transacional do sistema seja preservada. Em cenários adversos — como um cliente finalizando um pedido enquanto transita por uma área com instabilidade de rede móvel (3G/4G) —, a camada de transporte assegura que os pacotes fragmentados contendo os itens do carrinho e os dados de pagamento não sejam perdidos. Caso ocorra a perda de algum segmento no trajeto lógico até os servidores do Fornos, os temporizadores do TCP detectarão a ausência de confirmação e retransmitirão os pacotes automaticamente, garantindo que o pedido chegue de forma íntegra e sequencial à fila de processamento da padaria.

4 A CAMADA DE APLICAÇÃO: O PROTOCOLO HTTP NA PRÁTICA

Operando no topo da pilha TCP/IP, a Camada de Aplicação provê as interfaces de rede necessárias para que as regras de negócio do *software* interajam com o mundo externo. No desenvolvimento web moderno, o protocolo HTTP (*Hypertext Transfer Protocol*) é o padrão absoluto para essa comunicação. Operando sob o paradigma de requisição e resposta (*request-response*), o HTTP estrutura a troca de mensagens em blocos bem definidos, compostos primariamente por uma linha de requisição (contendo o método de ação e a URI), cabeçalhos (*headers*) para o tráfego de metadados, e um corpo (*body*) para o carregamento da carga útil (*payload*).

No ecossistema do Fornos, o HTTP atua como o maestro de todas as interações. A semântica dos métodos do protocolo é aplicada de forma rigorosa para refletir as intenções das operações na plataforma. Por exemplo, quando um cliente final acessa a URL do catálogo de uma padaria, o navegador emite uma requisição HTTP GET, instruindo o servidor a retornar apenas a leitura dos produtos, sem causar efeitos colaterais no estado do banco de dados. Em contrapartida, quando o gerente utiliza o painel administrativo para cadastrar um novo complemento ou quando o cliente finaliza o carrinho de compras, a aplicação (geralmente através das *Server Actions* do Next.js) dispara uma requisição HTTP POST. Nesse caso, os dados do formulário e do pedido são encapsulados no corpo da mensagem, frequentemente estruturados em formato JSON (*JavaScript Object Notation*).

Além do tráfego de dados transacionais, o protocolo da Camada de Aplicação desempenha um papel crítico na arquitetura de segurança da plataforma. O HTTP é, por natureza, um protocolo sem estado (*stateless*), o que significa que o servidor trata cada requisição como uma transação inteiramente nova e independente [3]. Para contornar essa limitação e implementar o rígido Controle de Acesso Baseado em Perfis (RBAC) do Fornos — que precisa diferenciar estritamente as permissões de um cliente comum das de um gerente —, o sistema faz uso

intensivo dos cabeçalhos HTTP para o gerenciamento de sessões.

Após a autenticação bem-sucedida de um administrador, o servidor devolve um *token* criptografado através do cabeçalho de resposta *Set-Cookie*. A partir desse momento, em todas as requisições subsequentes direcionadas às rotas administrativas, o navegador do gerente anexa automaticamente esse identificador no cabeçalho *Cookie* da requisição. Dessa forma, antes mesmo de o servidor processar consultas pesadas no banco de dados, o *middleware* da aplicação inspeciona o cabeçalho HTTP da requisição entrante. Se o *cookie* for inválido ou pertencer a um perfil sem privilégios, a conexão é imediatamente rejeitada com um código de erro 403 *Forbidden* ou 401 *Unauthorized*, provando como os metadados de rede são a primeira e mais rápida linha de defesa do sistema.

5 COMUNICAÇÃO INTERNA: SERVIDOR E BANCO DE DADOS

Embora o tráfego de dados na interface pública (cliente-servidor) seja a face mais visível da comunicação em aplicações web, a arquitetura de sistemas distribuídos exige que uma rede interna robusta opere nos bastidores. No projeto Fornos, a comunicação não se encerra quando a requisição HTTP alcança o servidor de hospedagem; ela desencadeia uma nova etapa de roteamento e estabelecimento de conexões na infraestrutura de *backend*, especificamente entre o servidor da aplicação em Next.js e o banco de dados relacional PostgreSQL.

Para garantir escalabilidade e padronização de ambiente, o Fornos utiliza o Docker como plataforma de containerização. Do ponto de vista de redes, conforme a documentação oficial da ferramenta [5], o *engine* do Docker atua criando uma infraestrutura de rede definida por software (SDN). Ao orquestrar os múltiplos serviços, a ferramenta cria uma rede do tipo *bridge* (ponte) virtual isolada, atribuindo endereços IPv4 privados dinamicamente para cada contêiner instanciado no hospedeiro. Essa abstração lógica permite que o servidor web e o banco de dados se descubram e se comuniquem através de uma sub-rede local, sem a necessidade de roteamento externo.

Na prática operacional, quando o sistema precisa persistir um novo pedido submetido por um cliente, o ambiente Node.js atua como um novo agente na camada de Transporte. A aplicação inicia um processo de *Three-Way Handshake* do TCP direcionado ao endereço IP interno do contêiner do PostgreSQL, utilizando, por padrão da tecnologia [6], a porta lógica 5432. Uma vez estabelecido o *socket* de comunicação, os comandos SQL e as tabelas de resposta trafegam encapsulados por essa conexão bidirecional confiável.

Essa topologia de rede interna isolada é um princípio fundamental de segurança em infraestruturas modernas. Ao restringir o roteamento de pacotes na porta 5432 exclusivamente ao tráfego originado pelo contêiner da própria aplicação, o ecossistema do Fornos bloqueia requisições vindas da internet mundial. Isso demonstra que as políticas de roteamento e o isolamento de redes IP (Camada 3) em conjunto com as portas lógicas (Camada 4) são tão cruciais para a proteção dos dados quanto as regras de autenticação implementadas na interface

visual.

6 ANÁLISE DE TRÁFEGO: HTTP POLLING E TRABALHOS FUTUROS COM WEBSOCKETS

No estágio atual de desenvolvimento e produção, a necessidade de atualizações em tempo real no sistema Fornos — como o alerta visual quando um novo pedido é recebido — é suprida através da técnica de *HTTP Polling* (ou *Short Polling*). Nessa abordagem, a interface de usuário (desenvolvida em React/Next.js) é programada para disparar requisições `HTTP GET` assíncronas ao servidor em intervalos de tempo pré-definidos e regulares, independentemente de haver ou não novos dados a serem consumidos.

Sob a perspectiva do tráfego de redes, o *Polling* tradicional é computacionalmente custoso e ineficiente em relação ao uso de banda. Cada requisição periódica exige o encapsulamento completo do protocolo HTTP, enviando dezenas ou centenas de bytes em cabeçalhos (incluindo *cookies* de autenticação, informações de *User-Agent* e tipos de conteúdo aceitos). Na esmagadora maioria das vezes, o servidor consulta o banco de dados e responde com um corpo vazio ou um código `304 Not Modified`, resultando em um alto volume de tráfego de controle (*overhead*) para a transmissão de nenhuma informação útil. Além disso, se a conexão não for mantida viva (via *Keep-Alive*), cada ciclo de *polling* pode exigir um novo *Three-Way Handshake* do TCP, aumentando a latência artificialmente.

Diante das limitações inerentes ao protocolo HTTP para comunicações bidirecionais contínuas, a implementação do protocolo WebSocket [7] está mapeada como a principal evolução arquitetural e trabalho futuro para a camada de redes do Fornos.

Diferentemente do modelo transacional de requisição e resposta do HTTP, o WebSocket inicia sua comunicação através de um *HTTP Upgrade*, solicitando ao servidor que a conexão TCP subjacente seja mantida permanentemente aberta. Uma vez estabelecido, o canal se torna *full-duplex* (bidirecional e simultâneo). Em uma futura versão da plataforma equipada com WebSockets, o navegador da padaria fará apenas uma única requisição ao abrir o painel. A partir desse momento, não haverá mais tráfego de requisições vazias; o próprio servidor terá a capacidade de empurrar (*Server Push*) os pacotes de dados para o cliente no exato milissegundo em que um novo pedido for finalizado no banco de dados, reduzindo drasticamente a carga na rede interna e a latência percebida pelo usuário final.

7 CONCLUSÕES

A concepção e o desenvolvimento de plataformas web modernas, embora fortemente abstraídas por *frameworks* de alto nível, são indissociáveis da infraestrutura de redes que os sustenta. Este estudo técnico evidenciou como os conceitos fundamentais de comunicação de dados, governados pela pilha TCP/IP, operam nos bastidores do sistema Fornos, garantindo a viabilidade, a segurança e a integridade de suas operações diárias.

Foi possível constatar que a confiabilidade da Camada de Transporte, provida pelo protocolo TCP e seus mecanismos de controle de conexão, é o pilar invisível que impede a perda de pacotes críticos durante as transações comerciais entre as padarias e seus clientes. Imediatamente acima, na Camada de Aplicação, o HTTP não atua apenas como um mero transportador de dados, mas como o próprio orquestrador das regras de negócio. O uso semântico de seus métodos e cabeçalhos sustenta não apenas o fluxo do sistema, mas também o rigoroso controle de acesso (RBAC) exigido pela plataforma.

O mapeamento da comunicação interna ressaltou, ainda, a importância das redes virtuais definidas por software. O isolamento de serviços provido pelo Docker demonstra que a integridade do banco de dados relacional depende de políticas estritas de roteamento e bloqueio de tráfego em portas lógicas específicas, reforçando que a segurança perimetral atua em múltiplas camadas.

Por fim, a análise do tráfego gerado pela técnica de atualizações contínuas (*HTTP Polling*) corrobora a premissa de que a engenharia de redes é um processo iterativo de otimização. A futura transição para o protocolo WebSocket representa o próximo passo arquitetural para mitigar o desperdício de banda e o excesso de processamento de controle.

Diante de todo o exposto, conclui-se que o conhecimento aprofundado sobre o fluxo de pacotes e o comportamento dos protocolos de rede é o verdadeiro diferencial entre a entrega de um *software* meramente funcional e a arquitetura de um sistema corporativo seguro, eficiente e verdadeiramente escalável.

Referências

- [1] TANENBAUM, Andrew S.; WETHERALL, David J. *Redes de Computadores*. 5. ed. São Paulo: Pearson Prentice Hall, 2011.
- [2] KUROSE, James F.; ROSS, Keith W. *Redes de Computadores e a Internet: uma abordagem top-down*. 6. ed. São Paulo: Pearson Education do Brasil, 2013.
- [3] MOZILLA DEVELOPER NETWORK (MDN). *An overview of HTTP*. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Overview>. Acesso em: jul. 2026.
- [4] FOROUZAN, Behrouz A. *Comunicação de Dados e Redes de Computadores*. 4. ed. São Paulo: McGraw-Hill, 2008.
- [5] DOCKER INC. *Docker Network Overview*. Disponível em: <https://docs.docker.com/network/>. Acesso em: jul. 2026.
- [6] POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL 16.3 Documentation*. Disponível em: <https://www.postgresql.org/docs/16/index.html>. Acesso em: jul. 2026.

- [7] FETTE, I.; MELNIKOV, A. *RFC 6455: The WebSocket Protocol*. Internet Engineering Task Force (IETF), 2011. Disponível em: <https://datatracker.ietf.org/doc/html/rfc6455>. Acesso em: jul. 2026.